UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE AREQUIPA
FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS

ESCUELA PROFESIONAL DE CIENCIA DE LA COMPUTACIÓN

ANGEL_DB: ANALIZADOR LÉXICO SQL

FASE 2 — PRÁCTICA DE LABORATORIO 4

Asignatura: Estructuras de Datos Avanzadas

Estudiantes

Condori Pallardel, Emilio Alejandro
Gama Llicahua, Fernando Antonio
Quispe Suarez, Angelo Josue

Perú – Arequipa
2026

Índice

1. Introducción **2**
1.1. Objetivo . . . 2
1.2. Alcance . . . 2

2. Especificación léxica **2**
2.1. Categorías de tokens . . . 2
2.2. Reglas léxicas . . . 2

3. Diseño e implementación **3**
3.1. Componentes . . . 3
3.2. Modelo de tokens . . . 3
3.3. Algoritmo de exploración . . . 4
3.4. Integración con el analizador sintáctico . . . 4

4. Manejo de errores **5**

5. Pruebas y validación **5**
5.1. Resultados . . . 5
5.2. Los siete escenarios . . . 5

6. Conclusiones **8**
6.1. Limitaciones y trabajo siguiente . . . 8

1 Introducción

Antes de que un gestor de base de datos pueda ejecutar una consulta, tiene que entender el texto en que está escrita. Ese trabajo se hace por etapas: primero se reconocen las palabras y los símbolos de la sentencia (análisis léxico), después se comprueba que formen una consulta válida (análisis sintáctico) y, por último, se construye el plan de ejecución.

Angel_DB es un servidor de base de datos relacional para sentencias `SELECT` simples que se construye en tres fases. La Fase 1, ya terminada, resolvió el almacenamiento de tuplas y su indexación con un árbol B+. Este informe corresponde a la primera parte de la Fase 2: el **analizador léxico**, que recibe una sentencia SQL como cadena de caracteres y devuelve la secuencia ordenada de tokens que luego usará el analizador sintáctico.

1.1 Objetivo

Diseñar e implementar un analizador léxico para un subconjunto de sentencias `SELECT`, capaz de transformar una cadena de entrada en una secuencia de tokens correctamente clasificados, informar los errores léxicos con su ubicación, y ofrecer una interfaz de salida adecuada para su integración con el analizador sintáctico.

1.2 Alcance

El analizador solo reconoce y clasifica lexemas. No ejecuta la consulta, no comprueba que las tablas o las columnas existan y no verifica la gramática: `SELECT FROM` produce los tokens `KEYWORD_SELECT`, `KEYWORD_FROM` y `EOF` sin ningún error, porque decidir si esa secuencia es una consulta válida corresponde a la etapa siguiente.

2 Especificación léxica

2.1 Categorías de tokens

Lexema	Tipo de token
`SELECT`, `FROM`, `WHERE`, `AND`, `OR`	`KEYWORD_SELECT`, `KEYWORD_FROM`, `KEYWORD_WHERE`, `KEYWORD_AND`, `KEYWORD_OR`
nombres de tablas y columnas	`IDENTIFIER`
`18`	`INTEGER_LITERAL`
`3.14`	`DECIMAL_LITERAL`
`'texto'`	`STRING_LITERAL`
`=`	`EQUAL`
`<>`, `!=`	`NOT_EQUAL`
`<`, `>`	`LESS`, `GREATER`
`<=`, `>=`	`LESS_EQUAL`, `GREATER_EQUAL`
`,`	`COMMA`
`;`	`SEMICOLON`
`*`	`ASTERISK`
fin de la entrada	`EOF`

Los nombres de los tokens son referenciales. Se incluyó `ASTERISK` para admitir `SELECT * FROM t`, que aparece en casi cualquier consulta real. `<>` y `!=` son el mismo token, y el lexema conserva cuál de los dos escribió el usuario.

2.2 Reglas léxicas

Mayúsculas y minúsculas.
Las palabras reservadas no distinguen mayúsculas: `select`, `Select` y `SELECT` son el mismo token. El lexema conserva lo que escribió el usuario.

Identificadores.
Empiezan con una letra ASCII o `_` y continúan con letras, dígitos o `_`.

Reservadas e identificadores.
Se lee la palabra completa y después se consulta la tabla de palabras reservadas. Por eso `selected`, `from_tabla` o `or_x` son identificadores aunque empiecen con una palabra reservada.

Números.
Un entero son dígitos; un decimal son dígitos, un punto y dígitos. No hay signo, notación científica ni punto inicial: `-5` y `.5` no son números del lexer, y el signo menos lo resolverá el analizador sintáctico.

Cadenas.
Van entre comillas simples; dentro de ellas, `''` representa una comilla. Pueden contener espacios, palabras reservadas, operadores y saltos de línea. El lexema conserva las comillas.

Operadores.
Se toma siempre el lexema más largo posible (*maximal munch*): en `a>=1` el `>=` es un solo token, mientras que `>=` con un espacio en medio son dos.

Espacios en blanco.
Espacio, tabulación, `\r` y `\n` separan tokens y se ignoran.

Posiciones.
Línea y columna empiezan en 1. La columna cuenta caracteres, no bytes: un carácter UTF-8 de varios bytes ocupa una sola columna.

Fin de entrada.
La lista de tokens siempre termina en `EOF`. Una entrada vacía produce solo `EOF`.

3 Diseño e implementación

3.1 Componentes

El analizador es un módulo independiente del resto del motor: vive en `include/minidb/lexer/` y `src/lexer/`, se compila como la biblioteca `minidb_lexer` y no usa la biblioteca de almacenamiento. Cada parte tiene una responsabilidad:

Componente	Responsabilidad
Modelo de tokens (`token.hpp`)	`TokenType`, y `Token` con tipo, lexema, línea y columna
Tabla de palabras reservadas (`keywords.cpp`)	Decide si una palabra es reservada, sin distinguir mayúsculas
Reconocedor léxico (`lexer.cpp`)	Recorre la entrada y construye los tokens y los errores
Manejo de errores	`LexicalError` con mensaje, línea y columna; el resultado agrupa tokens y errores
Interfaz de salida (`sql_lexer`)	Programa de línea de comandos que imprime los tokens en una tabla legible
Módulo de lectura de entrada	Lee la sentencia de un archivo, de la entrada estándar o de un texto dado con `-e`

3.2 Modelo de tokens

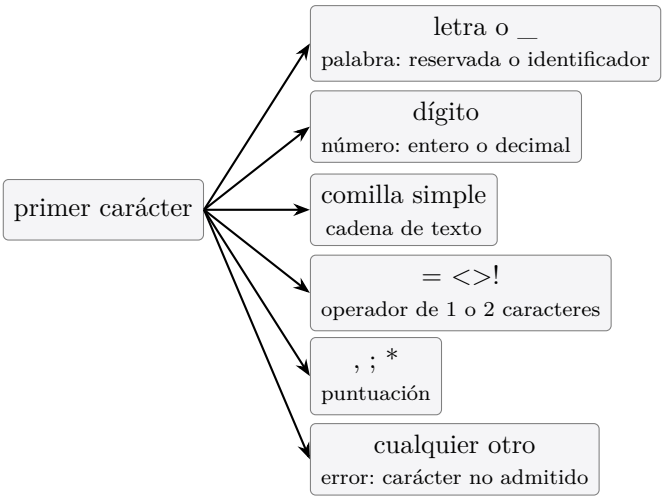
Cada token guarda su categoría, el texto exacto del lexema y la línea y la columna donde empieza. Con esos cuatro datos el analizador sintáctico puede decidir sin volver a mirar el texto original y señalar con precisión dónde está un error.

```
struct Token {
    TokenType type;        // categoria: kKeywordSelect, kIdentifier, ...
    std::string lexeme;    // texto tal como aparece en la entrada
    std::size_t line;      // desde 1
    std::size_t column;    // desde 1, cuenta caracteres
};

struct LexResult {
    std::vector<Token> tokens;         // en orden de aparicion, termina en EOF
    std::vector<LexicalError> errors;  // un lexema invalido nunca es token
};
```

3.3 Algoritmo de exploración

El reconocedor recorre la entrada una sola vez, de izquierda a derecha, manteniendo la posición actual y los contadores de línea y columna. En cada vuelta ignora los espacios en blanco y mira el *primer carácter* del siguiente lexema, que decide qué clase de token puede empezar ahí:



- **Palabras.** Se consume la secuencia de letras, dígitos y `_` y luego se consulta la tabla de palabras reservadas. Si está, el token es una palabra reservada; si no, un identificador.
- **Números.** Se consume toda la secuencia de letras, dígitos, `_` y puntos que empieza en un dígito y después se valida completa. Si son dígitos, es un entero; si son dígitos, un punto y dígitos, un decimal; en cualquier otro caso (`12abc`, `3.`, `1.2.3`) es un único número mal formado.
- **Cadenas.** Se consume hasta la comilla de cierre. Una comilla doble `"` seguida de más texto es una comilla escapada. Si se llega al final sin cerrar, es un error.
- **Operadores.** Tras leer el primer carácter se mira el siguiente para decidir entre `<`, `<=` y `<>`, o entre `>` y `>=`. Así un operador compuesto nunca se parte en dos tokens.

El costo es lineal en la longitud de la entrada: cada carácter se examina una vez.

3.4 Integración con el analizador sintáctico

La salida es una lista de tokens que conserva el orden original y termina en `EOF`. El analizador sintáctico no necesita acceder al texto ni al almacenamiento de tablas: recibe `LexResult` y decide si la secuencia cumple la gramática. Como el módulo no depende del resto del motor, puede probarse y reutilizarse por separado.

4 Manejo de errores

Un error léxico es una secuencia de caracteres que no forma ningún token válido. El analizador **no se detiene en el primero**: registra cada uno con su línea y su columna, sigue leyendo y reconoce con normalidad lo válido que los rodea. Así se informa de todos los problemas en una sola pasada.

Situación	Mensaje	Posición
Carácter que no pertenece a ningún token (`@`, `#`, `$`, `-`, `.`, ...)	`Caracter no admitido: 'x'`	el carácter
`!` que no forma `!=`	`Caracter no admitido: '!' (solo es valido como '!=')`	el `!`
Cadena sin comilla de cierre	`Cadena sin cerrar`	la comilla de apertura
Número seguido de letras, de un punto sin dígitos o con más de un punto	`Numero mal formado: '12abc'`	el primer dígito

La condición fundamental es que **un lexema inválido nunca se clasifica como token**. Por eso un número mal formado se informa como un único error que cubre toda la secuencia: si se partiera en trozos, aparecerían fragmentos (`12`, `abc`) que parecen tokens válidos.

5 Pruebas y validación

5.1 Resultados

Las pruebas se escriben con GoogleTest. El módulo tiene **49 pruebas** y todas pasan; junto con las 129 de la Fase 1 son 178 en total, y también pasan con AddressSanitizer y UBSan activados. El código compila sin advertencias con `-Wall -Wextra -Wpedantic -Wshadow -Wconversion -Wsign-conversion`.

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug -DMINIDB_ENABLE_SANITIZERS=ON
cmake --build build -j
ctest --test-dir build --output-on-failure      # 178 pruebas
./build/lexer_test                              # las 49 del analizador
```

Grupo	Pruebas	Qué verifica
`LexerTest`	35	tokens, orden, reservadas contra identificadores, enteros, decimales, cadenas, operadores, posiciones, errores
`LexerScenarios`	7	un test por cada escenario del enunciado
`LexerEdgeCases`	7	byte nulo, UTF-8, entrada grande, errores solamente, punto y signo menos
Total	**49**	

5.2 Los siete escenarios

Los escenarios se ejecutan con el programa `sql_lexer -e "..."`; el script `scripts/demo_lexer.sh` los corre todos seguidos.

1. Sentencia válida (el ejemplo del enunciado). Entrada: `SELECT nombre, edad / FROM estudiantes / WHERE edad >= 18;` en tres líneas.

```
 LINEA  COL  TIPO             LEXEMA
-------------------------------------------------
     1    1  KEYWORD_SELECT   SELECT
     1    8  IDENTIFIER       nombre
     1   14  COMMA            ,
     1   16  IDENTIFIER       edad
     2    1  KEYWORD_FROM     FROM
     2    6  IDENTIFIER       estudiantes
     3    1  KEYWORD_WHERE    WHERE
     3    7  IDENTIFIER       edad
     3   12  GREATER_EQUAL    >=
     3   15  INTEGER_LITERAL  18
     3   17  SEMICOLON        ;
     3   18  EOF              <EOF>

Tokens: 12 (incluye EOF)
```

La secuencia coincide con la del Cuadro 1 del enunciado: `KEYWORD_SELECT`, `IDENTIFIER`, `COMMA`, . . . , `GREATER_EQUAL`, `INTEGER_LITERAL`, `SEMICOLON`, `EOF`.

2. Operadores compuestos. Entrada: `a >= 1 AND b <= 2 OR c <>3 AND d != 4`. Los cuatro operadores salen como tokens individuales (`GREATER_EQUAL`, `LESS_EQUAL` y dos `NOT_EQUAL`):

```
 LINEA  COL  TIPO             LEXEMA
-------------------------------------------------
     1    1  IDENTIFIER       a
     1    3  GREATER_EQUAL    >=
     1    6  INTEGER_LITERAL  1
     1    8  KEYWORD_AND      AND
     1   12  IDENTIFIER       b
     1   14  LESS_EQUAL       <=
     1   17  INTEGER_LITERAL  2
     1   19  KEYWORD_OR       OR
     1   22  IDENTIFIER       c
     1   24  NOT_EQUAL        <>
     1   27  INTEGER_LITERAL  3
     1   29  KEYWORD_AND      AND
     1   33  IDENTIFIER       d
     1   35  NOT_EQUAL        !=
     1   38  INTEGER_LITERAL  4
     1   39  EOF              <EOF>

Tokens: 16 (incluye EOF)
```

3. Literales. Entrada: `SELECT 42, 3.14, 'Ana Quispe' FROM t;`

```
 LINEA  COL  TIPO             LEXEMA
-------------------------------------------------
     1    1  KEYWORD_SELECT   SELECT
     1    8  INTEGER_LITERAL  42
     1   10  COMMA            ,
     1   12  DECIMAL_LITERAL  3.14
     1   16  COMMA            ,
     1   18  STRING_LITERAL   'Ana Quispe'
     1   31  KEYWORD_FROM     FROM
     1   36  IDENTIFIER       t
     1   37  SEMICOLON        ;
     1   38  EOF              <EOF>

Tokens: 10 (incluye EOF)
```

4. Palabras reservadas e identificadores. Entrada: `select selection from fromage where wherever`. Las tres palabras reservadas se reconocen aunque estén en minúsculas; `selection`, `fromage` y `wherever`, que solo empiezan como una reservada, son identificadores.

```
 LINEA  COL  TIPO             LEXEMA
-------------------------------------------------
     1    1  KEYWORD_SELECT   select
```

```
     1    8  IDENTIFIER       selection
     1   18  KEYWORD_FROM     from
     1   23  IDENTIFIER       fromage
     1   31  KEYWORD_WHERE    where
     1   37  IDENTIFIER       wherever
     1   45  EOF              <EOF>

Tokens: 7 (incluye EOF)
```

5. Espacios en blanco. Las pruebas `WhitespaceIsIgnored` y `SpacesTabsAndNewlinesAreSkippedAndLines` comprueban que espacios, tabulaciones, `\r` y saltos de línea se ignoran, y que el contador de líneas avanza con cada `\n`. En el escenario 1 se ve en las columnas: cada línea reinicia la cuenta.

6. Errores léxicos. *Carácter no admitido.* Entrada: `SELECT a # FROM t;`. El `#` se informa en la columna 10 y no aparece como token; el resto se reconoce con normalidad.

```
 LINEA  COL  TIPO             LEXEMA
-------------------------------------------------
     1    1  KEYWORD_SELECT   SELECT
     1    8  IDENTIFIER       a
     1   12  KEYWORD_FROM     FROM
     1   17  IDENTIFIER       t
     1   18  SEMICOLON        ;
     1   19  EOF              <EOF>

Tokens: 6 (incluye EOF)

Errores lexicos (1):
  [1:10] Caracter no admitido: '#'
```

Cadena sin cerrar. Entrada: `SELECT * FROM t WHERE nombre = 'Ana`. El error apunta a la comilla de apertura, no al final de la entrada, y la cadena rota no se clasifica como literal.

```
 LINEA  COL  TIPO             LEXEMA
-------------------------------------------------
     1    1  KEYWORD_SELECT   SELECT
     1    8  ASTERISK         *
     1   10  KEYWORD_FROM     FROM
     1   15  IDENTIFIER       t
     1   17  KEYWORD_WHERE    WHERE
     1   23  IDENTIFIER       nombre
     1   30  EQUAL            =
     1   36  EOF              <EOF>

Tokens: 8 (incluye EOF)

Errores lexicos (1):
  [1:32] Cadena sin cerrar
```

Número mal formado. Entrada: `SELECT 12abc, 3. FROM t;`. Cada secuencia inválida es un solo error.

```
 LINEA  COL  TIPO             LEXEMA
-------------------------------------------------
     1    1  KEYWORD_SELECT   SELECT
     1   13  COMMA            ,
     1   18  KEYWORD_FROM     FROM
     1   23  IDENTIFIER       t
     1   24  SEMICOLON        ;
     1   25  EOF              <EOF>

Tokens: 6 (incluye EOF)

Errores lexicos (2):
  [1:8] Numero mal formado: '12abc'
  [1:15] Numero mal formado: '3.'
```

7. Casos límite. Entrada vacía: solo `EOF`. Sentencia incompleta (`SELECT nombre FROM`): el lexer clasifica lo que hay y termina con `EOF`, sin error, porque comprobar que la consulta esté completa es tarea del analizador sintáctico.

```
 LINEA  COL  TIPO             LEXEMA
-------------------------------------------------
     1    1  KEYWORD_SELECT   SELECT
     1    8  IDENTIFIER       nombre
     1   15  KEYWORD_FROM     FROM
     1   19  EOF              <EOF>

Tokens: 4 (incluye EOF)
```

Un token pegado al final de la entrada (`edad>=18`, sin espacio ni punto y coma) también se reconoce, y el `EOF` queda inmediatamente después.

6 Conclusiones

- El analizador reconoce y clasifica todas las categorías pedidas, más el asterisco, y conserva el orden y la posición (línea y columna) de cada token.
- Tomar siempre el lexema más largo evita partir los operadores compuestos, y consultar la tabla de palabras reservadas *después* de leer la palabra completa evita confundir `selection` con `select`.
- Los errores léxicos se informan con su ubicación sin detener el análisis, y un lexema inválido nunca se convierte en token. Los números mal formados se informan como un único error.
- Al ser un módulo independiente, con una interfaz pequeña (`Lexer::Tokenize` y `LexResult`), el analizador sintáctico de la siguiente etapa puede usarlo sin conocer nada del almacenamiento.

6.1 Limitaciones y trabajo siguiente

- Los identificadores aceptan solo letras ASCII, dígitos y `_`; una letra con tilde se informa como carácter no admitido.
- No hay números con signo ni notación científica; el signo menos lo resolverá el analizador sintáctico.
- El lexer no verifica la gramática. Esa comprobación, y la construcción del árbol de consulta, son la siguiente parte de la Fase 2.

Repositorio: `https://github.com/AngeloQs2004/SGDB/tree/fase_2_advance`